
Relatório Técnico de Sprint

Equipe: SIVIRA — Pesquisa Operacional

Sprint: 19

Período: 20/10/25 — 02/11/25

Responsável Técnico: Jardel Rodrigues

Data de Emissão: 02/10/25

1. Progressos (O que foi realizado na Sprint?)

Resumo Técnico:

- **Objetivo 01:** Sistema completo de alocação automática de funcionários com análise de conflitos temporais, validação de disponibilidade e rollback transacional.
- **Objetivo 02:** Interface de gestão de funcionários com visualização de agendas, carregamento automático de logs, extração de conflitos e relatórios filtráveis.
- **Objetivo 03:** Resolução completa dos 4 problemas técnicos identificados na Sprint 18, garantindo consistência transacional, rastreabilidade e sincronização entre módulos.

Atividades Concluídas:

- **Tarefa:** Implementação do Sistema Completo de Alocação Automática de Funcionários.
 - **Descrição técnica:** O sistema foi desenvolvido através da classe 'GestorFuncionarios', implementando padrão Singleton para garantir única instância em memória e prevenir inconsistências. A arquitetura centraliza o controle de pedidos já alocados através de estrutura de dados dedicada, permitindo validação eficiente contra dupla alocação. O carregamento de requisitos é automatizado através de detecção inteligente de arquivos no diretório de logs, extraindo metadados via expressões regulares que identificam atividades, janelas temporais, tipos profissionais permitidos e fatores de priorização. O algoritmo de alocação implementa estratégia multicritério considerando três dimensões principais: engajamento do funcionário no pedido atual (para manter continuidade), compatibilidade profissional baseada em FIPs (Fatores de Importância Profissional) da atividade e do funcionário, e disponibilidade temporal validada em múltiplas camadas. O sistema de validação verifica exhaustivamente folgas semanais e mensais configuradas, compatibilidade com turnos de trabalho (matutino, vespertino, noturno), intervalos obrigatórios de descanso entre jornadas, e conflitos de ocupação temporal com atividades já alocadas. A integridade transacional é garantida através de mecanismo de rollback automático que, ao detectar falha em qualquer atividade do pedido, reverte atômicamente

todas as alocações já registradas para aquele pedido, liberando os funcionários para novas alocações. O módulo de análise de conflitos gera relatórios detalhados identificando todos os funcionários compatíveis por tipo profissional, especificando para cada funcionário incompatível o motivo preciso da rejeição (folga, turno, intervalo ou conflito temporal), permitindo tomada de decisão informada. A rastreabilidade é assegurada através de logs estruturados segregados em diretórios de sucesso e erro, registrando todas as operações realizadas com detalhamento completo de funcionário, atividade, horários e resultado da alocação.

- **Responsável:** Jardel Rodrigues
- **Status:** Concluído
- **Observações:** Código versionado no repositório do GitHub.

- **Tarefa:** Desenvolvimento do Menu Completo de Gestão de Funcionários.

- **Descrição técnica:** A interface foi construída oferecendo conjunto abrangente de funcionalidades para gestão completa do ciclo de vida das alocações. O sistema de visualização de agendas organiza ocupações por funcionário, agrupando cronologicamente por dia e apresentando informações estruturadas incluindo horários de início e término, durações calculadas em formato legível, identificadores de ordem e pedido associados, e resumos consolidados de horas trabalhadas por período. A funcionalidade de carregamento automático reconstrói em memória o estado completo das alocações através de leitura e parsing dos arquivos de log, identificando registros bem-sucedidos e reconstruindo ocupações idênticas às originalmente registradas, permitindo continuidade de operação após reinicializações. O extrator de conflitos implementa análise automática dos logs identificando alocações falhadas através de marcadores específicos, gerando listas estruturadas para análise posterior e identificação de padrões de indisponibilidade. O gerador de relatórios oferece capacidade de filtragem precisa por ordem e pedido específicos, apresentando estatísticas detalhadas incluindo total de funcionários, quantidade de funcionários com e sem alocação, total de ocupações registradas, e identificação clara de recursos ociosos. As operações de manutenção implementam granularidade em dois níveis: limpeza seletiva de alocações de pedidos específicos sem afetar outros pedidos, e reset completo do sistema retornando todos os funcionários ao estado livre. O sistema incorpora proteção robusta contra dupla alocação através de verificação preventiva que bloqueia processamento de pedidos já alocados anteriormente. A funcionalidade de processamento em lote detecta automaticamente todos os arquivos válidos no diretório de requisitos, extrai metadados de ordem e pedido através de expressões regulares aplicadas aos nomes dos arquivos, e processa sequencialmente cada pedido gerando resumo estatístico final com contagem precisa de sucessos e falhas.
- **Responsável:** Jardel Rodrigues
- **Status:** Concluído
- **Observações:** Código versionado no repositório do GitHub.

- **Tarefa:** Resolução dos Problemas Técnicos Identificados na Sprint 18
 - **Descrição técnica:** Os quatro problemas críticos identificados foram completamente resolvidos através de refatorações arquiteturais específicas:
 - Problema 1 - Inconsistência entre Cancelamento e Validação:
Implementado sistema de arquivamento que move logs de pedidos cancelados para diretório dedicado 'logs/cancelados_manual/' ao invés de deletá-los. Esta abordagem preserva rastreabilidade completa mantendo histórico de todas as operações, permitindo auditorias posteriores, enquanto remove os logs do fluxo de validação ativo. O sistema mantém estrutura de metadados nos arquivos movidos, incluindo timestamp de cancelamento e motivo quando informado.
 - Problema 2 - Duplo Mecanismo de Rollback Não Coordenado:
Desenvolvida classe 'CoordenadorRollback' que centraliza e coordena todas as operações de rollback através de interface unificada. O coordenador implementa padrão de transação distribuída, garantindo atomicidade na liberação de equipamentos, funcionários e limpeza de logs. O mecanismo opera em duas fases: fase de preparação que verifica possibilidade de rollback em todos os gestores, e fase de commit que executa rollback coordenado apenas se todos os gestores confirmarem disponibilidade. Em caso de falha parcial, o sistema registra estado inconsistente e gera alerta para intervenção manual, prevenindo estados intermediários não detectados.
 - Problema 3 - Validação Ignora Almoxarifado:** Adicionado módulo 'validar_almoxarifado()' ao 'ValidadorPedidos', expandindo validação tripla original (equipamentos + funcionários) para quádrupla (equipamentos + funcionários + estados + almoxarifado). O validador verifica existência de comandas geradas para o pedido, confirma processamento pelo 'GestorAlmoxarifado', valida disponibilidade de estoque para todos os itens requeridos, e identifica itens críticos abaixo do estoque mínimo. Pedidos com falhas de almoxarifado são rejeitados automaticamente com motivos detalhados, prevenindo aprovação de pedidos sem garantia de matéria-prima.
 - Problema 4 - Dessincronia GerenciadorPedidos - ValidadorPedidos:
Implementado sistema de sincronização através de consulta prévia ao 'GerenciadorPedidos' antes de iniciar validação baseada em logs. O 'ValidadorPedidos' agora recebe referência ao 'GerenciadorPedidos' e verifica existência do pedido na memória antes de processá-lo. Pedidos removidos da memória mas ainda presentes em logs são automaticamente ignorados na validação, com registro em log de auditoria indicando dessincronia detectada e ação tomada. A sincronização bidirecional garante que apenas pedidos ativamente gerenciados sejam validados e exportados.
 - **Responsável:** Jardel Rodrigues
 - **Status:** Concluído
 - **Observações:** Código versionado no repositório do GitHub.
-

2. Planos (O que será feito na próxima Sprint?)

Objetivos Técnicos Prioritários:

- **Objetivo 01:** Sistema de Recuperação de Estado via Parsing de Logs de Equipamentos
 - **Descrição:** Desenvolver infraestrutura completa para recuperação automática do estado de equipamentos através da análise inteligente de logs de produção. O sistema implementará detecção e validação automática de arquivos de log disponíveis, extraindo metadados críticos como data de execução, identificadores de ordem e pedidos processados. A arquitetura incorporará conjunto especializado de parsers, um para cada categoria de equipamento do sistema (bancadas, fornos, batedeiras, masseiras, câmaras refrigeradas, freezers, fogões, balanças, misturadoras com cocção, fritadeiras, armários de fermentação, divisoras, modeladoras, embaladoras), cada um implementando lógica específica para extrair ocupações do formato de log correspondente. Os parsers realizarão extração estruturada de informações incluindo janelas temporais de ocupação, identificadores de ordem e pedido associados, descrições de atividades executadas, e itens produzidos. O sistema utilizará DTOs (Data Transfer Objects) especializados para estruturação imutável dos dados extraídos: 'OcupacaoDTO' encapsulando informações de uma ocupação individual, 'EstadoEquipamentoDTO' agregando todas as ocupações de um equipamento específico, e 'RelatorioRecuperacaoDTO' consolidando resultado completo da operação de recuperação. A validação estrutural verificará integridade dos logs incluindo presença de cabeçalhos obrigatórios, seções de equipamentos bem formadas, e estatísticas finais consistentes. O sistema oferecerá capacidade de ordenação temporal de logs, filtragem por ordem específica, e identificação do log mais recente para recuperação automática. Arquivo principal: 'utils/recuperacao/detector_logs.py'
- **Objetivo 02:** Restauração Automática de Objetos de Equipamentos em Memória
 - **Descrição:** Implementar camada de restauração que aplica estados extraídos dos logs diretamente nos objetos de equipamentos em memória, reconstruindo estado operacional completo. O sistema desenvolverá conjunto de restauradores especializados, um por categoria de equipamento, cada um conhecendo a estrutura interna específica do objeto correspondente e capaz de restaurar corretamente atributos de ocupação, cronogramas e estados internos. Os restauradores implementarão padrão de estratégia permitindo extensibilidade para novos tipos de equipamentos sem modificação de código existente. O orquestrador central 'RecuperadorEstado' coordenará todo o processo através de pipeline bem definido: detecção e validação do log, extração da lista de equipamentos presentes, iteração sobre cada equipamento selecionando parser apropriado para extração de ocupações, seleção de restaurador correspondente para aplicação no objeto, e validação final da consistência da recuperação. O sistema incorporará validadores especializados: 'ValidadorOcupacoes' verificando sobreposições

temporais e consistência de janelas, e 'ValidadorConsistencia' assegurando que estados recuperados são logicamente válidos e todos os equipamentos foram processados corretamente. A arquitetura oferecerá modo de operação dual: modo completo que aplica restauração nos objetos reais, e modo análise que apenas realiza parsing e validação sem modificar estado, útil para inspeção e debugging. O sistema gerará relatórios detalhados de recuperação incluindo estatísticas de equipamentos processados, total de ocupações recuperadas, erros encontrados por equipamento, e resumo consolidado com indicador de sucesso. A implementação garantirá rastreabilidade completa através de logging estruturado em cada etapa, permitindo auditoria de todas as operações de recuperação executadas. Arquivo principal: 'utils/recuperacao/recuperador_estado.py'

Atividades Planejadas:

- **Tarefa 01** - Implementar DetectorLogs com capacidade de detecção automática e extração de metadados via regex.
- **Tarefa 02** - Desenvolver validador estrutural de logs verificando integridade e completude.
- **Tarefa 03** - Criar DTOs imutáveis (OcupacaoDTO, EstadoEquipamentoDTO, RelatorioRecuperacaoDTO) para estruturação de dados.
- **Tarefa 04** - Implementar ParserBase abstrato definindo contrato para parsers especializados.
- **Tarefa 05** - Desenvolver 17 parsers especializados, um para cada tipo de equipamento do sistema.
- **Tarefa 06** - Implementar RestauradorBase abstrato definindo interface para restauradores.
- **Tarefa 07** - Criar 17 restauradores especializados com lógica específica por tipo de equipamento.
- **Tarefa 08** - Desenvolver ValidadorOcupacoes para verificação de consistência temporal.
- **Tarefa 09** - Implementar ValidadorConsistencia para validação de estados gerais.
- **Tarefa 10** - Criar RecuperadorEstado orquestrando pipeline completo de recuperação com geração de relatórios
- Recursos Necessários:
 - **Ferramentas:** VSCode + Python3.12 + Ferramenta de IA Anthropic Claude.
 - **Equipes:** SIVIRA — Pesquisa Operacional.

Critérios de Sucesso:

- **Objetivo 01** - Sistema de Recuperação de Estado via Parsing de Logs:
 - **1. Precisão na Detecção de Logs (100% de cobertura):**
 - Sistema deve detectar todos os arquivos .log válidos no diretório configurado
 - Extração de metadados (ordem, pedidos, data) deve ser 100% precisa para nomes padronizados

- Validação estrutural deve identificar logs corrompidos ou incompletos
 - Taxa de falsos positivos na validação: 0%
- **2. Completude na Extração de Dados:**
 - Parsers devem extrair 100% das ocupações registradas em logs bem formados
 - Cada ocupação extraída deve conter todos os campos obrigatórios (timestamps, ordem, pedido, atividade)
 - Timestamps devem ser parseados corretamente em objetos datetime
 - Erros de parsing devem ser capturados e reportados, não silenciosamente ignorados
- **3. Integridade dos DTOs:**
 - DTOs devem ser imutáveis após criação (garantir thread-safety)
 - Validações de campo obrigatório devem prevenir criação de DTOs inválidos
 - Propriedades calculadas (como total_ocupacoes) devem ser consistentes com dados armazenados
 - Serialização/desserialização de DTOs deve preservar todos os dados
- **4. Performance de Parsing:**
 - Extração de metadados via regex não deve gerar backtracking catastrófico
 - Uso de memória deve ser proporcional ao tamanho do log (não carregar tudo em memória)
- **5. Robustez e Recuperação de Erros:**
 - Sistema deve lidar graciosamente com logs parcialmente corrompidos
 - Falha no parsing de um equipamento não deve abortar processamento dos demais
 - Logs com encoding incorreto devem ser detectados e reportados
 - Sistema deve fornecer mensagens de erro acionáveis para correção
- **Objetivo 02 - Restauração Automática de Objetos de Equipamentos:**
 - **1. Precisão na Restauração (100% de fidelidade):**
 - Estado restaurado deve ser idêntico ao estado original que gerou o log
 - Todas as ocupações extraídas devem ser aplicadas nos objetos correspondentes
 - Atributos internos (cronogramas, disponibilidade) devem ser reconstruídos corretamente
 - Validação cruzada: re-gerar log do estado restaurado deve produzir log equivalente
 - **2. Cobertura de Tipos de Equipamentos:**
 - Sistema deve suportar restauração de todos os 17 tipos de equipamentos
 - Cada tipo deve ter restaurador especializado testado individualmente

- Equipamentos com estruturas complexas (ex: fornos multi-câmara) devem ser restaurados completamente
- Taxa de sucesso de restauração: $\geq 95\%$ quando logs são válidos
- **3. Consistência Pós-Restauração:**
 - ValidadorOcupacoes deve confirmar ausência de conflitos temporais
 - ValidadorConsistencia deve confirmar integridade de todos os estados
 - Nenhuma ocupação "fantasma" pode ser introduzida durante restauração
 - Nenhuma ocupação do log pode ser perdida durante restauração
- **4. Qualidade dos Relatórios:**
 - RelatorioRecuperacaoDTO deve conter estatísticas precisas e completas
 - Erros por equipamento devem ser específicos e acionáveis
 - Resumo final deve indicar claramente sucesso ou falha da operação
 - Formato do relatório deve ser legível para humanos e parseável por máquinas
- **5. Modos de Operação:**
 - Modo completo (aplicar restauração) deve modificar objetos corretamente
 - Modo análise (apenas parsing) não deve modificar nenhum estado
 - Sistema deve permitir rollback de restauração se validação final falhar
 - Orquestrador deve garantir atomicidade: ou restaura tudo ou nada
- **6. Integração com Sistema Existente:**
 - RecuperadorEstado deve integrar-se transparentemente com GestorProducao
 - Objetos restaurados devem ser utilizáveis imediatamente pelo sistema
 - Restauração não deve quebrar invariantes de negócio dos equipamentos
 - Sistema deve suportar recuperação incremental (adicionar ocupações sem limpar existentes)

3. Problemas (O que precisa ser resolvido?)

Desafios Técnicos:

- Não houveram problemas nessa Sprint.

Impedimentos:

- Não houveram impedimentos nessa Sprint.
-

4. Considerações Finais

- **Pontos Positivos:**

- **Resolução Completa de Débito Técnico:**

- A Sprint 19 focou primariamente em resolver os quatro problemas críticos identificados na Sprint anterior, demonstrando comprometimento com qualidade e manutenibilidade. Todas as inconsistências arquiteturais foram endereçadas através de refatorações bem planejadas que não introduziram novos problemas.

- **Consistência Transacional Garantida:**

- A implementação do 'CoordenadorRollback' estabeleceu padrão de coordenação centralizada para operações distribuídas, eliminando possibilidade de estados intermediários inconsistentes. Esta abordagem arquitetural pode ser replicada para outros cenários que exigem atomicidade.

- **Rastreabilidade Aprimorada:**

- O sistema de arquivamento de cancelamentos e a sincronização entre gestores e validadores asseguram que todas as operações sejam rastreáveis e auditáveis, cumprindo requisitos de governança e permitindo análises post-mortem detalhadas.

- **Validação Abrangente:**

- A expansão do 'ValidadorPedidos' para incluir almoxarifado completa a validação holística de pedidos, garantindo que aprovações ocorram apenas quando todos os recursos necessários (equipamentos, funcionários, materiais) estão disponíveis.

- **Lições Aprendidas:**

- **Importância do Endereçamento Proativo de Débito Técnico**

- Resolver problemas identificados imediatamente na sprint seguinte previne acumulação de débito técnico que posteriormente se torna exponencialmente mais custoso de remediar. A disciplina de dedicar uma sprint completa à resolução garantiu soluções robustas.

- **Coordenação é Fundamental em Arquiteturas Distribuídas**

- A necessidade do 'CoordenadorRollback' evidencia que sistemas com múltiplos gestores independentes requerem camada de coordenação explícita. Assumir que componentes independentes manterão consistência sozinhos é receita para bugs sutis e difíceis de reproduzir.

- **Sincronização Memória-Disco Requer Atenção Especial**

- A dessincronia entre 'GerenciadorPedidos' (memória) e 'ValidadorPedidos' (disco) demonstra que sistemas que mantêm estado em múltiplos locais precisam de estratégia clara de sincronização. A solução escolhida (memória como fonte de verdade, logs como registro histórico) estabelece hierarquia clara.

- **Validação Incremental Pode Ser Armadilha**

- A validação original que ignorava almoxarifado mostra risco de validações incompletas: pedidos passavam validação mas falhavam em produção por falta de material. Validações devem ser projetadas holisticamente desde o início, considerando todas as dependências do processo.

Anexos

- <https://github.com/jardel-ifce/sivira-src equip>
-